

TP Numero 3

Optimización de índices

Materia: Base de datos 2

Profesor: Carlos Yácomo

Integrantes del grupo C:

Matheo Bazan
Augusto Ingrassia
Uriel Vega
Elias Perez
Lautaro Mejias
Franco Squillo

Parte 1 — Poblar la base masivamente con datos generados por IA

Antes de poder medir una optimización, hace falta un volumen de datos realista. Los seeds de la Guía Docente cargan apenas un puñado de filas; para que un Seq Scan sea lento hace falta escala.

1. Pedir a la IA (OpenCode) un script SQL o un generador (por ejemplo, con `generate_series`) que cargue: al menos 50.000 productos distribuidos en las categorías existentes, 20.000 usuarios y 200.000 pedidos con sus detalles.

Prompt Pedido: Utilizando la información de “chema.sql” y “objects.sql” Necesito un Script SQL o un generador para pgAdmin que cargue: al menos 50.000 productos distribuidos en las categorías existentes, 20.000 usuarios y 200.000 pedidos con sus detalles.

Resultado del prompt:

```
BEGIN;

-- 1. Poblar Categoría (Inserta 5 categorías iniciales si la tabla está vacía)
INSERT INTO categoria (nombre, descripcion)
VALUES
  ('Comida Rápida', 'Hamburguesas, papas y combos'),
  ('Bebidas', 'Gaseosas, jugos y aguas'),
  ('Postres', 'Helados, tortas y dulces'),
  ('Pizzas y Empanadas', 'Pizzas variadas y empanadas artesanales'),
  ('Ensaladas', 'Opciones saludables y frescas')
ON CONFLICT (nombre) DO NOTHING;

-- 2. Insertar 50.000 productos respetando la restricción (stock > 0 cuando disponible = TRUE)
INSERT INTO producto (nombre, precio, descripcion, stock, disponible, categoria_id, eliminado)
SELECT
  'Producto Masivo ' || g AS nombre,
  ROUND((500 + (random() * 4500))::numeric, 2) AS precio,
  'Descripción del producto masivo número ' || g AS descripcion,
  floor(1 + (random() * 200))::int AS stock, -- Stock entre 1 y 200 para cumplir CHECK
  TRUE AS disponible,
  (g % 5) + 1 AS categoria_id, -- Distribuidos entre las 5 categorías existentes
  FALSE AS eliminado
FROM generate_series(1, 50000) AS g;

-- 3. Insertar 20.000 usuarios usando el ENUM 'ADMIN' / 'USUARIO' y columnas exactas
INSERT INTO usuario (nombre, apellido, mail, celular, contrasena, rol, eliminado)
SELECT
  'Nombre.' || g AS nombre,
  'Apellido.' || g AS apellido,
  'usuario.' || g || '@foodstore.com' AS mail,
  '54911' || floor(10000000 + (random() * 89999999))::text AS celular,
  'hash_seguro_contrasena_' || g AS contrasena,
  (CASE WHEN g % 50 = 0 THEN 'ADMIN' ELSE 'USUARIO' END)::rol AS rol,
  FALSE AS eliminado
FROM generate_series(1, 20000) AS g;

-- 4. Insertar 200.000 pedidos respetando ENUMs 'estado_pedido' y 'forma_pago'
INSERT INTO pedido (fecha, estado, total, forma_pago, usuario_id, eliminado)
SELECT
  CURRENT_DATE - (floor(random() * 365)::int) AS fecha,
  (ARRAY['PENDIENTE', 'CONFIRMADO', 'TERMINADO', 'CANCELADO'])[floor(1 + (random() * 4))::int]:estado_pedido AS estado,
  0.00 AS total, -- Se puede actualizar con los subtotales luego
  (ARRAY['TARJETA', 'TRANSFERENCIA', 'EFECTIVO'])[floor(1 + (random() * 3))::int]:forma_pago AS forma_pago,
  floor(1 + (random() * 19999))::int AS usuario_id,
  FALSE AS eliminado
FROM generate_series(1, 200000) AS g;

-- 5. Insertar detalles de pedido (1 a 3 items por pedido)
INSERT INTO detalle_pedido (cantidad, precio_unitario, subtotal, pedido_id, producto_id, eliminado)
SELECT
  cant AS cantidad,
  prec AS precio_unitario,
  ROUND((cant * prec)::numeric, 2) AS subtotal,
  p.id AS pedido_id,
  floor(1 + (random() * 49999))::int AS producto_id,
  FALSE AS eliminado
FROM pedido p
CROSS JOIN LATERAL (
  SELECT
    floor(1 + (random() * 5))::int AS cant,
    ROUND((500 + (random() * 4500))::numeric, 2) AS prec
) calc
CROSS JOIN LATERAL generate_series(1, floor(1 + (random() * 2))::int)
ON CONFLICT (pedido_id, producto_id) DO NOTHING; -- Evita duplicados en la restricción UNIQUE

COMMIT;

-- Actualización del optimizador para planes EXPLAIN reales
ANALYZE categoria;
ANALYZE producto;
ANALYZE usuario;
ANALYZE pedido;
ANALYZE detalle_pedido;
```

2. Leer línea por línea el script generado antes de ejecutarlo: verificar que respeta las restricciones (CHECK, UNIQUE, FK) y que no toca la base de producción.

3. Ejecutarlo dentro de una transacción, sobre una copia de la base del proyecto, con respaldo previo — protocolo de seguridad de la cátedra.

4. Correr ANALYZE sobre las tablas afectadas para que el optimizador actualice sus estadísticas antes de medir.

Parte 2 — Laboratorio: consultas lentas, EXPLAIN y optimización medida

Con la base ya poblada masivamente, se eligen consultas reales de queries.sql que se vuelven lentas a esta escala — por ejemplo, el listado de productos por categoría o el historial de pedidos por usuario sin índice — y se optimizan siguiendo el flujo profesional de la cátedra: medir antes, proponer, medir después, decidir con datos.

2.1 Consigna

1. Elegir al menos tres consultas de queries.sql (o variantes propias sobre el modelo) que tarden de forma perceptible sobre la base masiva.

Consultas elegidas:

- ♦ Consulta 1: Filtro y Ordenamiento en Productos por Categoría (Variante de [HU-PROD-01](#)

/B):

```
SELECT p.id, p.nombre, p.precio, p.stock, c.nombre AS categoria
FROM producto p
JOIN categoria c ON c.id = p.categoria_id
WHERE p.eliminado = FALSE
AND p.precio BETWEEN 1000 AND 3000
ORDER BY p.precio DESC;
```

- ♦ Consulta 2: Facturación por Categoría y Mes (Consulta B):

```
SELECT c.nombre AS categoria,
       date_trunc('month', ped.fecha) AS mes,
       SUM(dp.subtotal) AS facturado
FROM detalle_pedido dp
JOIN pedido ped ON ped.id = dp.pedido_id AND ped.eliminado = FALSE
JOIN producto pr ON pr.id = dp.producto_id
JOIN categoria c ON c.id = pr.categoria_id
WHERE dp.eliminado = FALSE
GROUP BY c.nombre, date_trunc('month', ped.fecha)
ORDER BY mes, facturado DESC;
```

- ♦ Consulta 3: Ranking de Usuarios por Gasto Acumulado (Consulta C):

```
SELECT u.id, u.nombre || ' ' || u.apellido AS usuario,
       SUM(ped.total) AS gasto,
       RANK() OVER (ORDER BY SUM(ped.total) DESC) AS puesto
FROM pedido ped
JOIN usuario u ON u.id = ped.usuario_id
WHERE ped.eliminado = FALSE
GROUP BY u.id, u.nombre, u.apellido
ORDER BY puesto;
```

2. Para cada una, correr EXPLAIN ANALYZE y guardar el plan completo (captura o texto) antes de cualquier cambio.

3. Pasarle a la IA el plan real obtenido y pedirle que proponga reescrituras y/o índices que lo mejoren, justificando cada propuesta en términos del propio plan (qué nodo ataca, por qué esperaría que cambie).

4. Leer línea por línea cada CREATE INDEX o reescritura propuesta antes de aplicarla. Rechazar toda propuesta que no se entienda por completo.

5. Aplicar los cambios aceptados sobre la copia de trabajo y volver a medir con EXPLAIN ANALYZE

♦ Consulta 1: Filtro y Ordenamiento en Productos por Categoría (Variante de HU-PROD-01 / B)

1. Análisis del Plan Original (ANTES de Optimizar)

Costo estimado: `cost=3436.68..3492.60`

Tiempo real de ejecución: `14.862 ms`

Nodos principales:

1. `Seq Scan on producto p`: Escanea las 50.000 filas de la tabla `producto` en memoria compartida (`shared hit=961`). Al no haber índice utilizable para el rango de precios, aplica el filtro `Filter: ((NOT eliminado) AND (precio >= 1000) AND (precio <= 3000))` descartando 27.626 filas (`Rows Removed by Filter`).
2. `Hash Join`: Une el resultado con la tabla `categoria`.
3. `Sort (quicksort)`: Realiza un ordenamiento explícito por `precio DESC` consumiendo tiempo adicional para ordenar las 22.374 filas filtradas.

2. Propuesta de Optimización

Para eliminar el `Seq Scan` y el nodo `Sort` (ordenamiento en memoria), vamos a crear un **índice compuesto B-Tree con inclusión de columnas (COVERING INDEX)** sobre la tabla `producto`:

```
CREATE INDEX idx_producto_precio_cubriente
ON producto (precio DESC)
INCLUDE (id, nombre, stock, categoria_id)
WHERE eliminado = FALSE;
```

¿Por qué debería mejorar este plan?

- Ordenamiento nativo: Al estar ordenado por `precio DESC` en la estructura del índice, el motor ya no necesita realizar el paso de `Sort (quicksort)`.
- Filtro parcial (`WHERE eliminado = FALSE`): Reduce el tamaño del índice al ignorar filas eliminadas.
- Coexistencia (`INCLUDE`): Permite resolver la consulta leyendo casi de forma directa el índice (`Index Scan` o `Index Only Scan`), evitando ir a las páginas de datos en disco.

3. Aplicación y Medición (DESPUÉS)

- Nodo principal: `Index Only Scan using idx_producto_precio_cubriente + Memoize + Index Scan using categoria_pkey`.
- Costo estimado: `cost=0.56..1807.17`
- Tiempo real de ejecución: `8.680 ms`

Mejora: ~1.71x más rápida (41.6% de reducción de tiempo real).

♦ Consulta 2: Facturación por Categoría y Mes (Consulta B)

Analizando la medición del Antes para nuestra Consulta 2 (Facturación por Categoría y Mes):

1. Análisis del Plan Original (ANTES de Optimizar)

Costo estimado: `cost=19005.91..19641.41`

Tiempo real de ejecución: `209.621 ms`

Nodos principales:

1. `Parallel Seq Scan` masivos: Para procesar la consulta, PostgreSQL recurrió a la paralelización en background (con 2 workers) ejecutando:
 - `Parallel Seq Scan on detalle_pedido dp`: Escanea las 400.000+ filas de detalles.
 - `Parallel Seq Scan on pedido ped`: Escanea las 200.000 filas de pedidos.
 - `Seq Scan on producto pr`: Escanea los 50.000 productos.
2. `Parallel Hash Join` repetidos: Realiza múltiples agrupamientos hash en memoria para enlazar los pedidos, productos y categorías.
3. `Incremental Sort & GroupAggregate`: Requiere ordenar y re-ordenar los datos agrupados por `date_trunc('month', ped.fecha)` y `SUM(dp.subtotal) DESC`.

2. Propuesta de Optimización

El cuello de botella está en los cruces entre `detalle_pedido` y `pedido` sobre la fecha del pedido y el estado de eliminación.

Para resolver los `Seq Scan` y acelerar el Join y la agrupación, crearemos dos índices clave:

1. Índice compuesto en `pedido`: Para acelerar el Join con `detalle_pedido` y la extracción de la fecha mes a mes.
2. Índice en `detalle_pedido`: Sobre la Foreign Key `producto_id` y `pedido_id` para optimizar el Hash Join con `producto`.

```
-- Índice 1: Cubre el join con detalle_pedido y la fecha de pedidos vigentes
CREATE INDEX idx_pedido_id_fecha_vigente
ON pedido (id, fecha)
```

```
WHERE eliminado = FALSE;
```

```
-- Índice 2: Acelera la asociación de detalles de pedidos hacia productos
```

```
CREATE INDEX idx_detalle_pedido_prod_ped
```

```
ON detalle_pedido (pedido_id, producto_id)
```

```
WHERE eliminado = FALSE;
```

3. Aplicación y Medición (DESPUÉS)

Al revisar la nueva ejecución, observamos que el tiempo total oscila alrededor de ~165.4 ms, sin mostrar cambios drásticos con respecto a la ejecución anterior.

¿Por qué ocurrió esto y cómo se podría resolver?

1. **Gasto de agregación global:** Como la consulta exige procesar **todos** los registros de la base masiva para calcular la facturación total agrupada por mes y categoría (sin ningún filtro **WHERE** que acorte el rango de fechas), PostgreSQL determina que leer secuencialmente los bloques en disco de forma paralela es más rápido que realizar millones de búsquedas dispersas en un árbol B-Tree (**Index Scan**).
2. **Optimizando la Estrategia:** Para que PostgreSQL realmente aproveche los índices y reduzca dramáticamente el costo de lectura en memoria, Tendríamos que crear un **Índice Cubriente Compuesto** que le permita obtener los datos que necesita usando **Index Only Scan** (sin ir a consultar la tabla principal):

♦ Consulta 3: Ranking de Usuarios por Gasto Acumulado (Consulta C)

Analizando la medición del **Antes** para nuestra **Consulta 3** (Ranking de Usuarios por Gasto Acumulado):

1. Análisis del Plan Original (ANTES de Optimizar)

Costo estimado: `cost=11839.65..11889.65`

Tiempo real de ejecución: `135.972 ms`

Nodos principales:

1. **Seq Scan on pedido ped:** Escanea secuencialmente los 200.000 pedidos en memoria descartando filas eliminadas (**Filter: NOT eliminado**).
2. **Hash Join & HashAggregate:** Realiza un join hash con la tabla **usuario** y luego agrupa en memoria/disco (**Batches: 5, Memory: 8241kB, Disk: 648kB**) por usuario para calcular el **SUM(ped.total)**.
3. **Sort explícito + WindowAgg:** Ordena el resultado de los 20.000 usuarios para aplicar la función de ventana **RANK() OVER (ORDER BY SUM(ped.total) DESC)** y realiza un último **Sort** por puesto.

2. Propuesta de Optimización

El cuello de botella está en el escaneo masivo de **pedido** y en el agrupamiento **HashAggregate** sobre las 200.000 filas para calcular la suma de los totales por usuario.

Crearemos un **Índice Compuesto Parcial** sobre la tabla `pedido`:

```
CREATE INDEX idx_pedido_usuario_total
ON pedido (usuario_id, total)
WHERE eliminado = FALSE;
```

¿Por qué debería mejorar este plan?

- **Index Only Scan / Aceleración de Joins:** Le permite al motor hacer un barrido directo sobre la estructura indexada buscando `usuario_id` y `total` sin consultar la tabla principal (**Heap**) para verificar totales.
- **Reducción de memoria en disco:** Al tener agrupados los datos de `usuario_id` en el índice, el agregador puede operar más rápido y reducir o eliminar los volcados a disco (**Disk Usage: 648kB**).

3. Aplicación y Medición (DESPUÉS)

Tiempo después: **109.490 ms**

Mejora: ~1.24x más rápida (19.4% de reducción en tiempo de ejecución).

Comportamiento del Optimización: El optimizador redujo el costo en el paso de escaneo y agrupamiento de los pedidos (pasando de **135.97 ms** a **109.49 ms**). Sin embargo, la consulta sigue requiriendo el nodo **WindowAgg** para la función de ventana **RANK() OVER (ORDER BY SUM(ped.total) DESC)** y un **Sort** final en memoria por puesto.

2.2 Tabla de resultados (incluir en la entrega, una fila por consulta)

Consulta	Plan antes (nodo, cost, tiempo real)	Cambio aplicado	Plan después (nodo, cost, tiempo real)	Mejora
1. Productos por Precio (Variante HU-PROD -01)	Seq Scan on producto + Sort (quicksort) Cost: 3436.68..3492.60 Tiempo: 14.86 ms	CREATE INDEX idx_producto_precio_cub riente ON producto (precio DESC) INCLUDE (id, nombre, stock, categoria_id) WHERE eliminado = FALSE;	Index Only Scan + Memoize (Sin Sort explícito) Cost: 0.56..1807.17 Tiempo: 8.68 ms	1.71x (41.6% más rápida)
2. Facturación por Categoría y Mes (Consulta B)	Parallel Seq Scan + Parallel Hash Join Cost: 19005.91..1964 1.41 Tiempo: 209.62 ms	CREATE INDEX idx_pedido_id_fecha_vig ente ON pedido (id, fecha) WHERE eliminado = FALSE; CREATE INDEX idx_detalle_pedido_prod _ped ON detalle_pedido (pedido_id, producto_id) WHERE eliminado = FALSE;	Parallel Seq Scan + Incremental Sort (El motor prioriza lectura secuencial distribuida para agregaciones globales) Cost: 19005.91..1964 1.41 Tiempo: 165.43 ms	1.26x (21.0% más rápida, variación marginal)
3. Ranking de Usuarios por Gasto (Consulta C)	Seq Scan on pedido + HashAggregate (con uso de disco) + WindowAgg Cost: 11839.65..11889 .65 Tiempo: 135.97 ms	CREATE INDEX idx_pedido_usuario_total ON pedido (usuario_id, total) WHERE eliminado = FALSE;	Seq Scan + HashAggregate + WindowAgg Cost: 11839.65..11889 .65 Tiempo: 109.49 ms	1.24x (19.4% más rápida)

Parte 3 — Lectura crítica de planes interpretados por IA

Un asistente de IA puede explicar en lenguaje natural qué hace un plan de EXPLAIN ANALYZE, pero esa explicación puede contener imprecisiones —por ejemplo, confundir un costo estimado con un tiempo real, o atribuir la mejora a la causa equivocada—. Esta parte entrena la lectura crítica de esas explicaciones.

1. Tomar uno de los planes reales obtenidos en la Parte 2 (con índice ya aplicado)
2. Pedirle a la IA que lo explique en lenguaje natural, nodo por nodo, sin mostrarle más contexto que el texto del plan.
3. Contrastar esa explicación contra el plan real, frase por frase, y marcar cualquier afirmación incorrecta o imprecisa (por ejemplo: “confunde cost con milisegundos”, “atribuye la mejora al índice equivocado”, “ignora el Rows Removed by Filter”).
4. Documentar los hallazgos en la tabla siguiente.

Afirmación de la IA	¿Correcta?	Corrección / Evidencia del plan real
"El proceso tardó 1807.17 milisegundos en leer la tabla."	No	Confunde el costo estimado en unidades de I/O (<code>cost=...1291.80 / 1807.17</code>) con tiempo real. El tiempo real del nodo fue <code>actual time=0.137..3.428 ms</code> .
"Luego ejecutó un QuickSort que demoró 0.56 milisegundos."	No	Afirma que se usó un QuickSort cuando en realidad el plan eliminó completamente el nodo Sort gracias a que el B-Tree ya mantenía las claves en orden <code>precio DESC</code> . Además, volvió a confundir el costo inicial <code>0.56</code> con milisegundos.
"Devolvió la respuesta en 8.68 segundos."	No	Error de escala de unidades. El tiempo real de ejecución (<code>Execution Time</code>) fue de <code>8.680 ms</code> (milisegundos), no segundos.
"Se utilizó un Index Only Scan con Heap Fetches: 0."	Sí	Afirmación correcta. El plan confirma <code>Heap Fetches: 0</code> , indicando que todos los datos necesarios fueron leídos directamente desde las páginas del índice sin tocar la tabla base.

Parte 4 — Consultas resumen y subconsultas bajo especificación precisa

Se trata de practicar el flujo de especificar con precisión —en lugar de pedir “hacé una consulta que...” de forma vaga— y de verificar que lo que la IA generó es equivalente a lo que se pidió, no solo que “corre sin error”.

1. Redactar una especificación precisa (spec) para al menos dos consultas sobre Food Store: una de resumen (agregación) y una con subconsulta. La especificación debe fijar explícitamente: tablas involucradas, filtro de borrado lógico, columnas de salida, orden y, si corresponde, criterio de corte (LIMIT, HAVING).

Spec 1 (Consulta de Resumen)

Especificación: Generar una consulta SQL sobre el esquema de Food Store que devuelva, para cada categoría vigente (`eliminado = FALSE`), el nombre de la categoría y la cantidad total de productos vigentes que posee (`eliminado = FALSE`), incluyendo las categorías sin productos con valor 0. Ordenar de mayor a menor cantidad de productos y, en caso de empate, alfabéticamente por nombre de categoría.

Spec 2 (Consulta con Subconsulta)

Especificación: Obtener el ID, nombre, apellido y mail de todos los usuarios vigentes (`eliminado = FALSE`) que hayan realizado al menos un pedido cuyo monto total sea estrictamente superior al promedio general de todos los pedidos vigentes registrados en la plataforma. Ordenar por ID de usuario ascendente.

2. Pedirle a la IA que genere el SQL a partir de esa especificación, sin mostrarle una solución previa.

Version A generada sin mostrarle solución previa del Spec 1:

```
SELECT c.nombre AS categoria,
       COUNT(p.id) AS total_productos
FROM   categoria c
LEFT JOIN producto p ON p.categoria_id = c.id AND p.eliminado = FALSE
WHERE  c.eliminado = FALSE
GROUP BY c.id, c.nombre
ORDER BY total_productos DESC, c.nombre ASC;
```

Version A generada sin mostrarle solución previa del Spec 2:

```
SELECT u.id, u.nombre, u.apellido, u.mail
FROM   usuario u
WHERE  u.eliminado = FALSE
AND    u.id IN (
    SELECT ped.usuario_id
    FROM   pedido ped
    WHERE  ped.eliminado = FALSE
    AND    ped.total > (SELECT AVG(total) FROM pedido WHERE eliminado = FALSE)
) ORDER BY u.id ASC;
```

3. Escribir, de forma independiente, una segunda versión propia (o pedirle a la IA una alternativa con distinta estructura: subconsulta vs. join, por ejemplo) que responda la misma pregunta.

Version B generada del spec 1:

```
SELECT c.nombre AS categoria,
       (SELECT COUNT(p.id)
        FROM producto p
        WHERE p.categoria_id = c.id AND p.eliminado = FALSE) AS total_productos
FROM categoria c
WHERE c.eliminado = FALSE
ORDER BY total_productos DESC, c.nombre ASC;
```

Version B generada del spec 2:

```
SELECT u.id, u.nombre, u.apellido, u.mail
FROM usuario u
WHERE u.eliminado = FALSE
AND EXISTS (
    SELECT 1
    FROM pedido ped
    WHERE ped.usuario_id = u.id
    AND ped.eliminado = FALSE
    AND ped.total > (SELECT AVG(total) FROM pedido WHERE eliminado = FALSE)
)
ORDER BY u.id ASC;
```

4. Verificar la equivalencia de resultados entre ambas versiones — por ejemplo, comparando el conteo de filas y una muestra de valores, o con EXCEPT entre ambos resultados.

Verificacion de equivalencia Spec 1:

```
-- Si son 100% equivalentes, ambas operaciones retornan 0 filas
(
    SELECT c.nombre AS categoria, COUNT(p.id) AS total_productos
    FROM categoria c LEFT JOIN producto p ON p.categoria_id = c.id AND p.eliminado = FALSE
    WHERE c.eliminado = FALSE GROUP BY c.id, c.nombre
)
EXCEPT
(
    SELECT c.nombre AS categoria, (SELECT COUNT(p.id) FROM producto p WHERE p.categoria_id = c.id AND p.eliminado = FALSE) AS total_productos
    FROM categoria c WHERE c.eliminado = FALSE
);
```

Verificación de equivalencia Spec 2:

```
(
  SELECT u.id, u.nombre, u.apellido, u.mail
  FROM usuario u WHERE u.eliminado = FALSE AND u.id IN (
    SELECT ped.usuario_id FROM pedido ped WHERE ped.eliminado = FALSE AND
    ped.total > (SELECT AVG(total) FROM pedido WHERE eliminado = FALSE)
  )
)
EXCEPT
(
  SELECT u.id, u.nombre, u.apellido, u.mail
  FROM usuario u WHERE u.eliminado = FALSE AND EXISTS (
    SELECT 1 FROM pedido ped WHERE ped.usuario_id = u.id AND ped.eliminado =
    FALSE AND ped.total > (SELECT AVG(total) FROM pedido WHERE eliminado = FALSE)
  )
);
```

Parte 5 — Competencia de optimización entre equipos

Consigna Común de la Competencia

- Consulta asignada: Búsqueda y listado paginado de productos vigentes por rango de precio, filtrando por categoría activa y ordenando de mayor a menor precio.
- Objetivo: Lograr el menor tiempo de ejecución real (*actual time* de **EXPLAIN ANALYZE**)

```
-- Consulta lenta fijada para la competencia
SELECT p.id, p.nombre, p.precio, p.stock, c.nombre AS categoria
FROM producto p
JOIN categoria c ON c.id = p.categoria_id
WHERE p.eliminado = FALSE
  AND c.eliminado = FALSE
  AND p.precio BETWEEN 500 AND 3500
ORDER BY p.precio DESC
LIMIT 100;
```

Bitácora de Estrategias y Pruebas Evaluadas por el Equipo

1. Estrategia 1 (Descartada - Propuesta inicial de IA):

- *Prueba:* Creación de un índice simple B-Tree únicamente sobre **producto(precio)**.
- *Resultado:* El optimizador siguió haciendo un **Seq Scan + Sort** explícito porque necesitaba evaluar **eliminado = FALSE** y buscar el resto de columnas en la tabla principal (**Heap**).
- *Decisión:* Descartada por no reducir significativamente el costo de I/O.

2. Estrategia 2 (Aceptada y Ganadora):

- *Prueba:* Creación de un Índice Cubriente Parcial Compuesto (**COVERING INDEX**) sobre **producto** con inclusión de columnas (**INCLUDE**) y filtro de estado lógico.

```
CREATE INDEX idx_competencia_producto_optimo
ON producto (precio DESC)
INCLUDE (id, nombre, stock, categoria_id)
WHERE eliminado = FALSE;
```

Resultado: El motor transformó la consulta a un **Index Only Scan** con **Heap Fetches: 0**. Como el índice ya almacena las filas en orden **precio DESC**, eliminó el paso de **Sort** por completo y la combinación con **c.id** vía **Index Scan** por Primary Key permitió resolver el **LIMIT 100** en milisegundos.

Decisión: Aceptada y consolidada.

Equipo	Estrategia Aplicada	Tiempo Antes (ms)	Tiempo Después (ms)	Mejora (x)
Nuestro Equipo (Grupo 3)	Índice Cubriente Parcial (precio DESC) con INCLUDE (id, nombre, stock, categoria_id) filtrado por eliminado = FALSE. Elimina el nodo Sort y realiza Index Only Scan.	18.42 ms	0.28 ms	65.78x (98.4% más rápida)

Declaración de Uso de IA (DUIA)

Herramienta	Para qué se usó	Prompt/Spec (resumen)	Se aceptó / se descartó y por qué
OpenCode / IA	Carga Masiva (Parte 1)	Utilizando lá información de “chema.sql” y “objects.sql” Necesito un Script SQL o un generador para pgAdmin que cargue: al menos 50.000 productos distribuidos en las categorías existentes, 20.000 usuarios y 200.000 pedidos con sus detalles.	Aceptado. Se verificó sintaxis y restricciones relacionales antes de ejecutar en bloque dentro de una transacción.
Kiro / IA	Propuesta de Índices (Parte 2)	"Analizá el plan EXPLAIN ANALYZE de la consulta de filtro por precio y proponé un índice que elimine el nodo Sort."	Aceptado. Se probó el índice cubriente (INCLUDE) y redujo el tiempo real de 14.86 ms a 8.68 ms .
Kiro / IA	Interpretación de Planes (Parte 3)	"Explica en lenguaje natural el plan de ejecución de la Consulta 1."	Evaluado críticamente. Se detectaron imprecisiones donde confundió costos estimados con milisegundos y afirmó falsamente la presencia de un nodo Sort.
OpenCode / IA	Generación de Spec y SQL (Parte 4)	"Generá consulta de facturación/producto s por categoría con subconsulta correlacionada."	Aceptado. Se verificó la equivalencia de resultados formalmente frente a la versión propia utilizando EXCEPT .